FitTrack — Gym Management System

Complete System Architecture, Code Flow & Technology Documentation

FitTrack is a fully functional, production-ready React web application designed for modern gym management, localized specifically for the Indian audience (INR ₹ currency, +91 mobile format, DD/MM/YYYY dates). The application covers complete gym workflow management including admin authentication, member management, trainer rosters, session scheduling, pricing plans, payment tracking, and equipment maintenance.

Core Frontend & Build Tools

• React.js (v19.2.8): Functional components + hooks only (useState, useEffect, useContext). All components use .jsx extension.
• Vite (v8.2.2): Next-generation fast frontend bundler & HMR development server.
• JavaScript (ES6+): Modern JS with async/await, array methods, destructured state.

UI & Styling

• Bootstrap 5 (v5.3.8): Full responsive grid, utility classes, dynamic tables, and cards.
• Bootstrap Icons (v1.13.1): SVG vector icons for UI actions and navigation.
• Custom CSS (index.css): Responsive theme variables (#00f3ff primary accent), high-contrast badges, light/dark theme styles.

Routing & State Management

• React Router DOM (v7.18.3): Client-side SPA routing with PrivateRoute guard.
• Context API (React): Global state management for Authentication (AuthContext) and Theme (ThemeContext).

HTTP Client & Backend Simulation

• Axios (v1.20.0): Centralized HTTP client configured with baseURL (axiosInstance.js).
• JSON Server (v0.17.4): REST API mock server reading/writing db.json database.
• React-Toastify (v11.1.0): Animated toast notifications for operations (Create, Edit, Delete, Errors).

Code Quality & Tooling

• ESLint (v8.57.0) & Oxlint: Code quality enforcement with 0 max-warnings rule.
• Concurrently (v10.0.5): Dual execution runner for JSON Server and Vite client.

A. Authentication & Security

• Login page (Login.jsx) validates email & password against /admins endpoint in db.json via Axios.
• Session persisted in localStorage & managed globally via AuthContext.
• PrivateRoute blocks unauthorized access to protected routes.

B. Members Directory (Members.jsx)

• Full Name limit: Restricted to max 20 characters (maxLength=20 + validation rule) with character counter.
• Email Validation: Enforces standard regex pattern (user@domain.com).
• Calendar Date Pickers: HTML5 date picker for Join Date and Expiry Date (toISODateString & fromISODateString).
• Device Image Upload: Local device file picker converting images to Base64 data URLs with instant preview.

C. Fitness Trainers (Trainers.jsx)

• Assigned Members Limit: Maximum 20 members limit with visual capacity warning.
• Experience Bounds: Numeric validation enforced between 1 year (min) and 50 years (max).
• Valid Email & Device Photo Upload: Image picker with custom fallback avatar.

D. Membership Plans (Plans.jsx)

• Plan creation with duration (1, 3, 6, 12 Months) and feature lists.

• Price Bounds: Enforced price range between min ¹1,000 and max ¹20,000 (INR).

E. Fee & Payments (Payments.jsx)

• Auto-Locked Amount: Read-only amount field automatically synced to selected plan price.
• Calendar Date Picker: Date selection converted to DD/MM/YYYY format.
• Two-Way Sync: Saving payment updates member name, plan, join date, and calculates membership expiry date (calculateExpiryDate).

F. Gym Equipment (Equipment.jsx)

• Stock quantity tracking with min 0 and max 10 units limit.
• Maintenance tracking with native date picker for last maintenance date.

G. Training Sessions & Booking System (Sessions.jsx)

• Monday to Saturday Schedule: Interactive day tabs and search filters.
• Gym Operating Hours: Enforced between 06:00 AM and 10:00 PM.
• Clock Time Pickers: Native time pickers for Start Time and End Time.
• Duration Limits: Enforced session duration between 1 hour (minimum) and 3 hours (maximum).
• Member Booking System: Booking slot allocation, capacity tracking, and booking cancellation.

The FitTrack application follows a clean 3-tier client-side architecture:

1. Presentation Layer: React Functional Components (.jsx) rendered with Bootstrap 5.
2. State & Utility Layer: Context API (Auth + Theme), formatters.jsx (INR formatting, Indian phone validation, date converters), and custom hooks.
3. Service & API Layer: Centralized axiosInstance executing REST calls to JSON Server (db.json).

Data Flow Sequence:
User Input -> Local State Handler -> Validation -> Axios Request -> JSON Server db.json -> State Update & Toast Notification -> UI Re-render.